Assignment 3: Handwritten Digit Classification on MNIST

Topics: Supervised Learning, Model Comparison, Hyperparameter Tuning

---

Objective

In this assignment, you will explore and implement six foundational Machine Learning algorithms, i.e., Logistic Regression, Naïve Bayes, k-Nearest Neighbors (kNN), Decision Tree, Support Vector Machines (SVM), and Multi-Layer Perceptron (MLP), for multi-class image classification. You will evaluate their performance on the benchmark MNIST dataset, analyze execution efficiency, and critically compare their strengths, weaknesses, and decision boundaries.

Dataset setup

- **Dataset source:** MNIST Handwritten Digit Dataset (28×28 grayscale images across 10 classes, representing digits 0 to 9).
- **Data splitting:** Use the official split or sample 60,000 images for training and 10,000 images for testing.
- **Preprocessing:**
  - Flatten 28×28 pixel matrices into 784-dimensional vectors.
  - Optional: Normalize pixel values from $[0, 255]$ to $[0.0, 1.0]$ or standard normal distribution ($\mu = 0, \sigma = 1$).

Task instruction You must construct six distinct model pipelines *from scratch* (you are encouraged to compare your custom implementations against `scikit-learn` results) and configure their hyperparameters for optimal performance:

1. **Model 1: Logistic regression (Multinomial / Softmax)**
   - *Configuration:* Implement L_2 regularization with an optimal reguralization strenth.
2. **Model 2: Naïve Bayes (Gaussian or Bernoulli)**
   - *Configuration:* Implement Gaussian Naïve Bayes for continuous pixel intensities or binarized Bernoulli Naïve Bayes (thresholding pixel intensities at $\theta = 0.5$).
3. **Model 3: k-nearest neighbors (kNN)**
   - *Configuration:* Experiment with $k \in \{3, 5, 7\}$ using Euclidean (L_2) or Manhattan (L_1) distance metrics.

4. **Model 4: Decision tree**

   - *Configuration:* Train using max-gain splitting criteria. Limit `max_depth` to prevent overfitting on noisy pixel features.

5. **Model 5: Support Vector Machine**

   - *Configuration:* Compare linear kernel SVM against radial basis function (RBF) kernel SVM with tuned penalty C and kernel coefficient γ.

6. **Model 6: Multi-Layer Perceptron**

   - *Configuration:* Construct a neural network with 1–2 hidden layers with ReLU/sigmoid activation, Softmax output layer, trained using Adam or SGD with momentum).

Evaluation criteria & required deliverables

Your submission will be assessed on four core pillars:

1. Submission & from-scratch implementation

- **Deliverable format:** Submit a single executed Jupyter Notebook following the strict naming convention: `Assignment-3-Student-Name.ipynb`.
- **Core algorithms:** All six algorithms (Logistic Regression, Naïve Bayes, kNN, Decision Tree, SVM, and MLP) must be coded **entirely from scratch** using basic vector/matrix operations (`numpy`). Using high-level estimator calls from libraries like `scikit-learn` for core model logic is strictly prohibited (except for optional benchmark comparison).

2. Quantitative & qualitative evaluation

- **Quantitative evaluation:** Display clear summary tables (Pandas DataFrames / Markdown tables) inside the notebook reporting Accuracy, Precision, Recall, Macro F_1-Score, Training Time (seconds), and Test Latency across all six custom models.
- **Qualitative evaluation:** Identify, display, and analyze 3–5 misclassified digit images per model. Discuss the specific visual ambiguities, stroke anomalies, or boundary cases causing the misclassifications.

3. Visualization deliverables

- **Performance charts:** Render comparative bar plots showing F_1-Score trade-offs and runtime speed across all models.
- **Confusion matrices:** Generate side-by-side 10×10 confusion matrices for all six models on the test set directly within the notebook output cells.

4. Discussion & critical analysis

- **Linear vs. Non-linear boundary dynamics:** Critically analyze how linear algorithms perform on high-dimensional raw pixel inputs compared to non-linear models.
- **Computational complexity & inference latency:** Contrast training time vs. inference latency trade-offs (e.g., non-parametric lazy learning in kNN vs. parametric models like MLP and SVM).
- **Hyperparameter impact:** Discuss how hyperparameter tuning (C, γ, tree depth, learning rate) impacted generalization and prevented overfitting.